

Visual Design Reuse Through Style Recognition and Transfer

Jeremy Warner
jeremy.warner@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

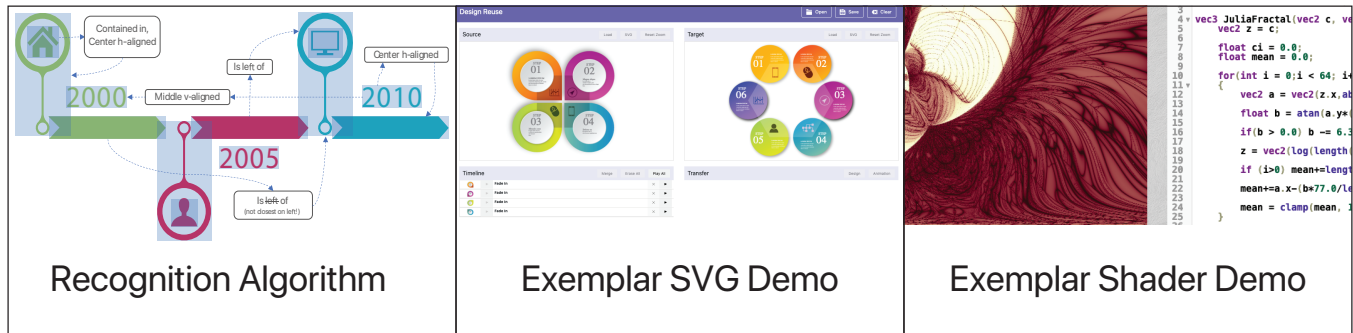


Figure 1: Overview of concepts from visual redesign through implicit patterns: flexible pattern recognition for vector graphics, and implicit style recognition and transfer for both 2D and 3D visual designs.

ABSTRACT

This work aims to transfer design attributes and styles within and across visual documents such as slides, graphic designs, and non-photorealistic renderings. Consistent style across elements is a hallmark of good graphic design. Many visual stylistic design patterns exist throughout visualizations, presentations, and interactive media experiences (games, visual novels). These patterns often exist in visual style guides, brand guides, and concept art. However, except for structured document layouts (e.g., HTML/CSS), design tools often do not enforce consistent style decisions or must be manually maintained. Synchronizing style guides and designs require significant effort, discouraging exploration and the mixing of new ideas. This work introduces algorithms that recognize implicit patterns and structures in visual documents along with interfaces that let designers operate on these patterns, specifically, to view and apply design changes across pattern instances flexibly. The key benefits of visual redesign through implicit patterns are: (1) removing any dependence on upfront formal style declarations, (2) enabling extraction and distribution of implicit visual patterns, and (3) facilitating the exploration of novel visual design concepts through the mixing of styles.

CCS CONCEPTS

• **Computing methodologies** → **Computer graphics**; *Image manipulation*; *Graphics systems and interfaces*.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '21 Adjunct, October 10–14, 2021, Virtual Event, USA

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8655-5/21/10.

<https://doi.org/10.1145/3474349.3477591>

KEYWORDS

style transfer, graphics, programming by example

ACM Reference Format:

Jeremy Warner. 2021. Visual Design Reuse Through Style Recognition and Transfer. In *The Adjunct Publication of the 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21 Adjunct)*, October 10–14, 2021, Virtual Event, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3474349.3477591>

1 INTRODUCTION

This work focuses on facilitating the reuse and transfer of structured visual documents and programs. Graphic designs like infographics, user interface sketches, and explanatory diagrams often contain multiple sets of elements with similar visual structures. In some cases, we also see elements with consistent visual structure across multiple different designs. Designers need to apply consistent edits such as style changes, layout adjustments, and animation effects across these repeating sets of elements. Such edits are tedious to perform manually, especially as the design size increases. One solution is to explicitly specify the structure of the design upfront and leverage this to transfer edits across sets of elements that share the predefined structure. For example, Microsoft PowerPoint's template slide feature allows users to edit the appearance of multiple slides at once. Similarly, popular UX design tools (e.g., Adobe XD, Figma) encourage users to define symbols or components that control the properties of repeated instances within a design, such as buttons, icons, or banners. Defining this structure initially requires substantial forethought, which can conflict with designers' workflows. In many cases, designers create and iterate on the whole graphic to get the overall design right before thinking of repeated elements, shared structure, or what edits to apply. A process for retroactively mixing and transferring previously designed styles

across visual documents provides a flexible alternative to formalized CSS or component-based guides.

2 RELATED WORKS

Related work falls into interactive graphics editors, context-aware application behavior, and machine learning approaches for visual style transfer. Citrine is an early example of inferring the context of a computer's clipboard content to be useful in more scenarios [8]. Xia et al. explored techniques that use implicit similarities within visual documents to enhance element selection [10] and object drawing and creation [10]. These demonstrations mainly focused on touch-based drawing applications though the interaction techniques and primitives extend beyond touch-first interfaces. Along with this idea of leveraging the structure of visual documents, Hoarau et al. show the potential of linking related objects to facilitate editing [4]. Sketch-n-sketch constructs a bi-directional relationship for providing direct manipulation on vector graphics [3]. This application is fascinating because the code serves as a way to generate images, and the images serve as a way to modify or generate code. Outside of advances within interactive graphical editors, machine-learning-based approaches also show promise for visual style transfer. Chen et al. showed how to recognize timeline structures from pixel images to augment the extension and generation of timeline info-graphics [1]. Inferring structure from visual diagrams automatically provides strong results, but omits additional information integrated into visual documents (e.g., layers, groups). Fisher et al. demonstrate using graphs to model the contents of a 3D scene, enabling context-specific search and other applications [2]. Park et al. showed that GANs (generative adversarial networks) could identify and transfer structure and texture between images [5]. Reddy et al. demonstrate a method that can identify instances of a pattern when given a base pattern image definition [6]. This promising work shows the bright future that automatic techniques will have when working creatively with visual media. Still, the technique operates specifically on a grid of pixels and omits other relevant implicit information present in visual documents.

3 RECOGNIZING IMPLICIT VISUAL PATTERNS

I previously researched methods to recognize hierarchical similarities between arbitrary vector graphic documents [7]. Machine-learning based approaches can work well to identify compositions in pixel-based images of graphic designs. Additional metadata exists in the context of the working editor, which can augment AI-based approaches. Specifically, most visual design tools contain much richer information than a grid of pixels. Vector graphics can scale to an infinitely high resolution and may often contain metadata like groups or layers. A 3D graphical rendering often contains object assets and shader effects that affect each pixel's rendering. Restricting the basis of techniques for style recognition and transfer to ML techniques on pixel grids is unnecessarily limiting.

We proposed a different approach to help designers apply edits consistently across a design. Instead of explicitly asking users to structure their content ahead of time, users provide an arbitrary set of *source* elements, apply the desired edits, and then automatically transfer the edits to a different collection of *target* elements. In this

workflow, the system is responsible for retroactively inferring the shared structure between the source and target elements. Thus, our approach can be applied to any vector graphics design, regardless of how it was created. Our method is not limited to graphics that share identical structures or elements but is flexible enough to accommodate common variations between the source and target graphics, such as color, shape, arrangement, or element cardinality. Moreover, it generalizes to different types of edits such as style transfer, layout adjustments, and applying animation effects.

A core challenge in realizing this approach was finding correspondences between the source and target graphics such that designers can apply the appropriate source edits to each target element. First, both the source and target graphics may contain many similar elements, both related and unrelated. Second, the target graphics may have strong stylistic and structural differences compared to the source graphics. The type and range of these differences vary for each design, making it hard to define a consistent matching algorithm based on heuristics. Finally, some types of edits must take into account nesting and ordering relationships between the elements. Transferring an edit properly elements requires identifying the analogous hierarchical structure for other graphical elements in the design by computing multi-level correspondence. Although designers may group elements during the creation process, these are not necessarily reliable indicators of perceptual structure and do not always correspond to the desired hierarchy for edit or transfer. These groups typically don't encode element ordering, which is important for temporal effects like animation.

The main contribution of this work is an automatic algorithm for determining the shared implicit visual patterns between source and target elements that address these challenges. Our method is based on graph kernels. Given a set of source and target graphics, we compute *relationship graphs* that encode the structure of the elements. We then analyze the source and target relationship graphs using graph kernels to compute element-wise correspondences. We also introduce an efficient method to hierarchically cluster and sequence the target elements into ordered trees whose structure is consistent with the source graphics. Together, the correspondences and ordered trees can transfer any edits effectively from the source to target elements.

4 ONGOING WORK: VECTOR GRAPHIC VISUAL REDESIGN EXEMPLAR

Currently, I am developing an application to demonstrate style and animation transfer for vector graphics (SVGs). Style in this sense covers static attributes which are straightforward to compare and differentiate (e.g., border, font, fill). The style also covers more complex animation patterns like those present in many presentations and user interface workflow mocks. We use the kernel-based hierarchical construction and comparison technique [7] as a foundation to demonstrate style transfer within a functional exemplar system for designers. Specifically, we are interested in research questions based on how users can effectively transfer visual styles. We are also exploring ways to explore alternatives best when the transfer method returns many ambiguous design options.

Leveraging a match of elements across canvases, we can accurately detect graphic matches and infer common structures between

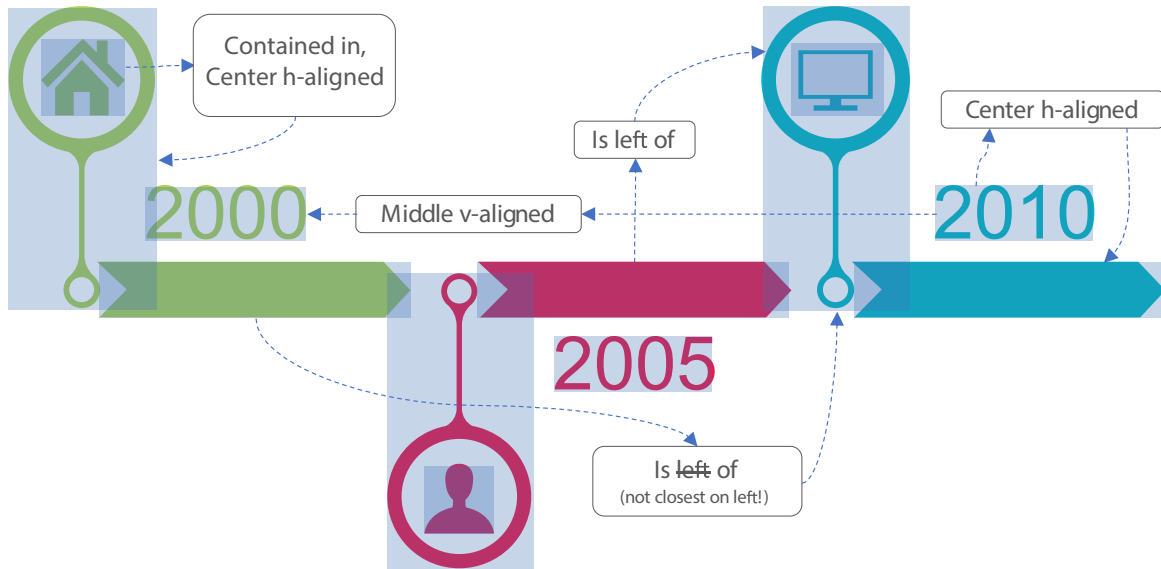


Figure 2: An example relationship graph for a set of elements. Only a subset of the edges is shown.

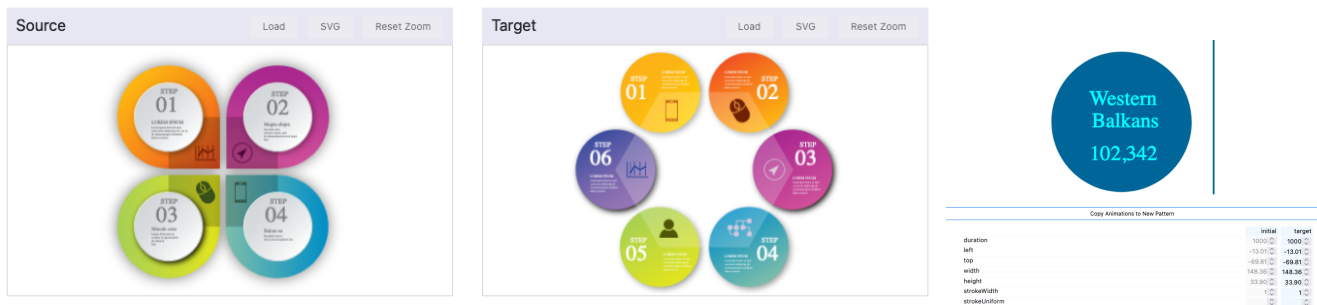


Figure 3: Transferring attributes and behaviors between documents. Viewing and customizing attributes to change between canvases.

graphics. This matching is often enough to correctly transfer basic styles corresponding to a style guide, similar to what would be encoded in a style guide. Specifically, we look at the transfer of coordinated animations or motion patterns in the context of hierarchical vector graphics. If the top transfer results are incorrect, we want to collect more context for re-ranking transfer options so that the system shows designers useful options. Users may help the algorithm out by providing an example mapping between canvases, augmenting the current algorithm. Other feedback also bolsters the pool of examples that the algorithm can leverage to refine its ability to match elements and hierarchies correctly. We are developing and testing this application with many SVG examples.

5 PROPOSED WORK: VISUAL REDESIGN FOR GRAPHICAL SHADERS

I plan to develop this idea of style transfer with more rendering techniques (shaders). The big idea here is that there is a rich space for exploring a set of styles of rendering that are analog to that of vector graphics. Where vector graphics have static attributes

like font or fill, 3D renderings will use object properties, position, and programmatic behavior. Websites like ShaderToy are a great place to explore new rendering design techniques. These interfaces have both rich structural programmatic metadata and visual output. The function-based structure and variance of shaders often make combining visual styles and aspects difficult. Similar to how Sketch-N-Sketch [3] linked the visual output with the generated program, I will draw connections between the code and visual output. Designers can use these connections to distill themes and styles of a shader and facilitate the mixing and transfer of styles between shaders.

6 DISSERTATION STATUS AND IMPACT

In the fall of 2021, I will start my 6th Ph.D. year in the EECS Department at UC Berkeley. I expect feedback from the consortium to help me refine the research methodology in the last milestones of developing and assessing the proposed system. I anticipate the remaining work to take 1-1.5 years. My research contributes new fundamental

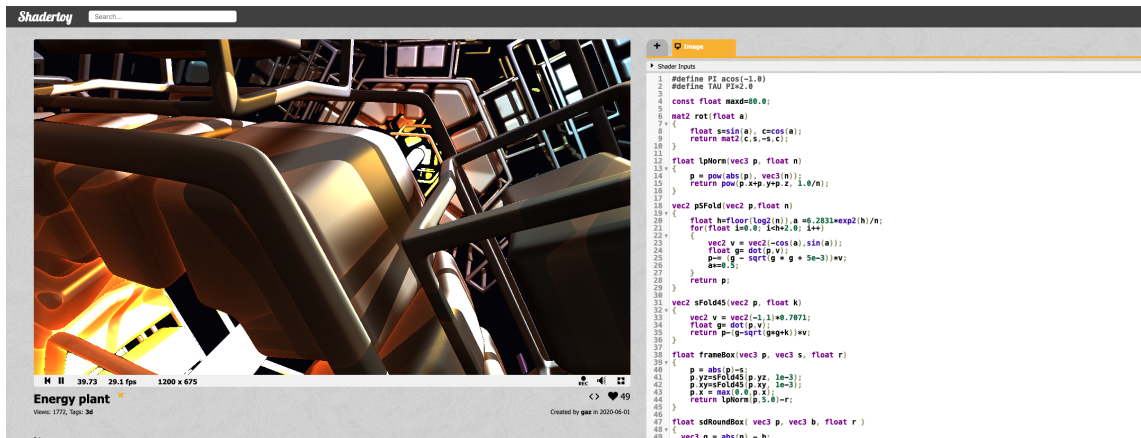


Figure 4: The ShaderToy [9] interface is a rich library of graphical programming examples, though determining why a section of the animation or scene is constructed a certain way is challenging; feedback on what part of the effect can be obfuscated by the code. Also, combining examples from the site can be difficult due to different shader semantics.

ideas and techniques in the fields of interactive graphics editors and style transfer across both vector graphics and shaders.

REFERENCES

- [1] Zhutian Chen, Yun Wang, Qianwen Wang, Yong Wang, and Huamin Qu. 2019. Towards automated infographic design: Deep learning-based auto-extraction of extensible timeline. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 917–926.
- [2] Matthew Fisher, Manolis Savva, and Pat Hanrahan. 2011. Characterizing Structural Relationships in Scenes Using Graph Kernels. In *ACM SIGGRAPH 2011 Papers* (Vancouver, British Columbia, Canada) (SIGGRAPH '11). Association for Computing Machinery, New York, NY, USA, Article 34, 12 pages. <https://doi.org/10.1145/1964921.1964929>
- [3] Brian Hempel, Justin Lubin, and Ravi Chugh. 2019. Sketch-n-Sketch: Output-Directed Programming for SVG. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*. 281–292.
- [4] Raphaël Hoarau and Stéphane Conversy. 2012. Augmenting the scope of interactions with implicit and explicit graphical structures. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1937–1946.
- [5] Taesung Park, Jun-Yan Zhu, Oliver Wang, Jingwan Lu, Eli Shechtman, Alexei A. Efros, and Richard Zhang. 2020. Swapping Autoencoder for Deep Image Manipulation. In *Advances in Neural Information Processing Systems*.
- [6] Pradyumna Reddy, Paul Guerrero, Matt Fisher, Wilmot Li, and Niloy J. Mitra. 2020. Discovering Pattern Structure Using Differentiable Compositing. *ACM Trans. Graph.* 39, 6, Article 262 (Nov. 2020), 15 pages. <https://doi.org/10.1145/3414685.3417830>
- [7] Hijung Valentina Shin, Jeremy Warner, Bjorn Hartmann, Celso F Gomes, Holger Winnemöller, and Wilmot Li. 2020. Multi-level Correspondence via Graph Kernels for Editing Vector Graphics Designs. In *Graphics Interface 2021*.
- [8] Jeffrey Stylos, Brad A Myers, and Andrew Faulring. 2004. Citrine: providing intelligent copy-and-paste. In *Proceedings of the 17th annual ACM symposium on User interface software and technology*. 185–188.
- [9] User:ChristinaCoffin. 2015. Shadertoy Rendering Example. <https://www.shadertoy.com/view/4l23zc>
- [10] Haijun Xia, Bruno Araujo, Toví Grossman, and Daniel Wigdor. 2016. Object-oriented drawing. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 4610–4621.